

Phase 1: Requirement Discovery & Traceability

Project Name: UniOrder – University Ordering Management System

Technology Stack: FastAPI (Python), Next.js, MySQL, JWT Authentication

1. Project Overview

UniOrder is a web-based ordering management system for universities. It allows university staff and laboratory assistants to request products such as office supplies, electronics, and lab equipment.

The system supports:

- User registration and login.
 - Product and category management.
 - Product browsing and searching.
 - Shopping cart management.
 - Order placement.
 - Order status tracking.
 - Admin order processing.
 - Inventory management.
-

2. Actor Classification

2.1 Primary Actors

Staff Member (Requester)

University employee, teaching assistant, or lab assistant who requests products.

This actor initiates the core business flow.

Use Cases: Register, login, browse products, search, add to cart, place order, track order.

Administrator

Responsible for managing products, inventory, and orders.

Initiates all management and operational workflows.

Use Cases: Manage categories, manage products, update stock, process orders, update order status.

2.2 Supporting Actors

Database Management System (MySQL)

Stores all persistent data.

Provides data storage services.

JWT Authentication Service

Creates and validates access tokens.

Supports secure authentication.

2.3 Offstage Actors

Department Manager

Reviews delivered products and budgets but does not directly use the system.

University Finance Department

Uses order records for accounting and reimbursement.

Indirect stakeholder affected by order data.

3. Functional Requirements

ID	Functional Requirement
FR1	User can register an account
FR2	User can log in using JWT authentication
FR3	Admin can manage categories
FR4	Admin can manage products
FR5	User can browse/search/filter products
FR6	User can add/update/remove items in cart
FR7	User can place an order
FR8	User can view order history
FR9	User can track order status
FR10	Admin can view all orders
FR11	Admin can update order status
FR12	Admin can manage inventory stock
FR13	System prevents ordering more than available stock

4. Sprint Coverage (Sprint 0–6)

Sprint	Features Delivered
Sprint 0	Authentication and initial product creation
Sprint 1	Category management and product catalog
Sprint 2	Shopping cart
Sprint 3	Checkout and order history
Sprint 4	Order status tracking
Sprint 5	Admin order management
Sprint 6	Inventory management

5. Traceability Heatmap

The traceability heatmap ensures every requirement is implemented and linked to at least one actor and sprint.

Requirement	Description	Primary Actor	Supporting Actor	Sprint
FR1	User registration	Staff Member	JWT Service, DB	0
FR2	User login	Staff Member	JWT Service, DB	0
FR3	Category CRUD	Administrator	DB	1
FR4	Product CRUD	Administrator	DB	0–1
FR5	Product browsing/search	Staff Member	DB	1
FR6	Cart management	Staff Member	DB	2
FR7	Place order	Staff Member	DB	3
FR8	Order history	Staff Member	DB	3
FR9	Track order status	Staff Member	DB	4
FR10	View all orders	Administrator	DB	5
FR11	Update order status	Administrator	DB	4–5
FR12	Inventory management	Administrator	DB	6
FR13	Stock validation	System	DB	2, 3, 6

6. Persona Discovery – AI User Avatar

Persona: Omar – Frustrated and Malicious Student Worker

Omar is a student assistant in a university lab. He is impatient, tries unusual inputs, and intentionally attempts to break the system.

Persona Goals

- Place orders quickly.
 - Try invalid or excessive quantities.
 - Access restricted admin pages.
 - Submit incomplete data.
 - Exploit security weaknesses.
-

Hidden Requirements and Edge Cases

Hidden Requirement 1: Prevent Ordering More Than Available Stock

Scenario: Omar adds 100 units while only 5 are in stock.

Requirement: The system must reject quantities greater than current stock.

Reasoning: Prevents negative inventory and invalid orders.

Hidden Requirement 2: Prevent Duplicate Registration Emails

Scenario: Omar registers twice with the same email address.

Requirement: Email must be unique.

Reasoning: Prevents account duplication and authentication issues.

Hidden Requirement 3: Block Unauthorized Access to Admin Endpoints

Scenario: Omar manually navigates to `/admin/orders`.

Requirement: Non-admin users must receive 403 Forbidden.

Reasoning: Protects sensitive data and operations.

Hidden Requirement 4: Handle Session Expiration Gracefully

Scenario: Omar's JWT token expires during checkout.

Requirement: Redirect to login and preserve intended action if possible.

Reasoning: Improves user experience and security.

Hidden Requirement 5: Prevent Checkout with Empty Cart

Scenario: Omar clicks checkout with no items.

Requirement: Checkout must be blocked.

Reasoning: Avoids invalid order records.

Hidden Requirement 6: Prevent Negative or Zero Quantities

Scenario: Omar submits quantity = 0 or -3.

Requirement: Quantities must be positive integers.

Reasoning: Maintains data integrity.

Hidden Requirement 7: Handle Product Deletion Referenced by Existing Orders

Scenario: Admin deletes a product already included in past orders.

Requirement: Preserve historical order item data.

Reasoning: Ensures reporting and audit accuracy.

7. Hidden Non-Functional Requirements

ID	Requirement
NFR1	Passwords must be securely hashed
NFR2	APIs should respond within 2 seconds for standard requests
NFR3	UI must be responsive on desktop and mobile
NFR4	All protected routes require authentication
NFR5	Inputs must be validated server-side

8. Use Case to Requirement Mapping

Use Case	Related Requirements
Register	FR1
Login	FR2
Manage Categories	FR3
Manage Products	FR4
Browse Products	FR5
Manage Cart	FR6
Checkout	FR7, FR13
View Order History	FR8
Track Order	FR9
View All Orders	FR10
Update Order Status	FR11
Manage Inventory	FR12
